MQTT LED Control

Using ESP32 & HiveMQ Cloud

IoT Project Report

Embedded Systems

Author: Swati

Platform: ESP32 + ESP-IDF | Broker: HiveMQ Cloud

1. Project Overview

This project demonstrates IoT-based LED control using an ESP32 microcontroller connected to the HiveMQ Cloud MQTT broker. The system allows remote ON/OFF control of an external LED via MQTT messages sent from any MQTT client (e.g., HiveMQ Web Client).

The ESP32 connects to Wi-Fi, establishes a secure TLS connection to HiveMQ Cloud, subscribes to the topic "esp32/led", and toggles an LED on GPIO 4 based on received messages.

2. Project Demo & Output

[Image: project_demo.png not found - place it in the project folder]

Figure: Working setup showing ESP32 on breadboard with blue LED, ESP-IDF terminal displaying MQTT messages, and HiveMQ Web Client used to publish ON/OFF commands.

Project Output Video:

https://youtu.be/OKMco-J3JLo?si=sgc1bR76ot9SNDn_

3. Hardware Setup

Component	
ESP32 DevKit	Main microcontroller (Wi-Fi enabled)
External LED (Blue)	Connected to GPIO 4
Resistor (220 Ohm)	Current limiting resistor for LED
Breadboard	For circuit connections
Jumper Wires	For wiring components

Circuit Connections:

- GPIO 4 -> Resistor (220 Ohm) -> LED Anode (+)
- LED Cathode (-) -> GND

4. Software Architecture

Communication Flow:

```
+----------------+       +---------------------+       +-----------+
```

```
|  MQTT Client   |--TLS->| HiveMQ Cloud Broker |--TLS->|   ESP32   |
| (Web/App/CLI)  |       |  (port 8883)        |       |  (GPIO 4) |
+----------------+       +---------------------+       +-----+-----+
                                                             |
                                                            LED
```

Step-by-step flow:

- 1. ESP32 connects to Wi-Fi network
- 2. Establishes secure MQTT connection (TLS) to HiveMQ Cloud
- 3. Subscribes to topic: esp32/led
- 4. User publishes "ON" or "OFF" from any MQTT client
- 5. ESP32 receives message and toggles LED accordingly

5. Key Configuration

Parameter	
Wi-Fi SSID	Swati_Hotspot
MQTT Broker	HiveMQ Cloud (TLS, port 8883)
MQTT Topic	esp32/led
LED GPIO Pin	GPIO 4
Security	TLS with ESP-IDF certificate bundle
QoS Level	1 (At least once delivery)
Authentication	Username/Password

6. Software Components

ESP-IDF Modules Used:

Module	
esp_wifi	Wi-Fi STA mode connection
mqtt_client	MQTT client (publish/subscribe)
esp_crt_bundle	TLS certificate verification
driver/gpio	GPIO control for LED
nvs_flash	Non-volatile storage initialization
freertos	Task scheduling and delays

Project Structure:

```
MQTT_LED_HiveMQ/
|-- CMakeLists.txt       (Project-level build config)
|-- sdkconfig            (ESP-IDF configuration)
|-- main/
|   |-- CMakeLists.txt   (Component build config)
|   +-- main.c           (Application source code)
+-- README.md
```

7. Functional Description

LED Initialization:

- GPIO 4 configured as output (no pull-up/pull-down)
- LED starts in OFF state

Wi-Fi Connection:

- Connects in STA (Station) mode

- Auto-reconnects on disconnection
- Logs IP address upon successful connection

MQTT Communication:

- Connects to HiveMQ Cloud using MQTTS (port 8883)
- Username/password authentication
- TLS secured via ESP-IDF trusted certificate bundle
- Subscribes to "esp32/led" with QoS 1

LED Control Logic:

Message	
ON	LED turns ON (GPIO HIGH)
OFF	LED turns OFF (GPIO LOW)
Other	Warning: "Unknown command"

8. Serial Monitor Output

Sample output from ESP-IDF monitor during operation:

```
I (MQTT_LED): Starting MQTT LED project...
I (MQTT_LED): Connecting to Wi-Fi...
I (MQTT_LED): Wi-Fi started
I (MQTT_LED): Got IP: 192.168.x.x
I (MQTT_LED): Starting MQTT...
I (MQTT_LED): MQTT client started
I (MQTT_LED): MQTT CONNECTED
I (MQTT_LED): Subscribed to topic: esp32/led
I (MQTT_LED): MQTT SUBSCRIBED, msg_id=20325
I (MQTT_LED): MQTT DATA RECEIVED
Topic: esp32/led
Data: ON
I (MQTT_LED): LED -> ON
I (MQTT_LED): MQTT DATA RECEIVED
Topic: esp32/led
Data: OFF
I (MQTT_LED): LED -> OFF
```

9. Build & Flash Instructions

```
# Set ESP-IDF environment
. ~/esp/esp-idf/export.sh

# Build the project
idf.py build

# Flash to ESP32
idf.py -p /dev/ttyUSB0 flash

# Monitor serial output
idf.py -p /dev/ttyUSB0 monitor
```

10. Tools & Technologies

Tool / Technology	
ESP-IDF	v5.x (Espressif IoT Dev Framework)
Microcontroller	ESP32 DevKit
MQTT Broker	HiveMQ Cloud (Free Tier)
Protocol	MQTT over TLS (MQTTS)

IDE	VS Code + ESP-IDF Extension
Language	C
RTOS	FreeRTOS

11. Features

- Secure MQTT communication over TLS
- Remote LED control from anywhere via internet
- Auto Wi-Fi reconnection on disconnection
- Clean event-driven architecture
- HiveMQ Cloud free tier compatible
- Minimal build configuration for fast compilation
- Error handling with detailed ESP logging

12. Conclusion

This project successfully demonstrates IoT-based remote control of a hardware peripheral (LED) using the MQTT protocol with cloud connectivity. The ESP32 securely connects to HiveMQ Cloud and responds to real-time commands, showcasing a practical IoT publish-subscribe pattern.

This architecture can be extended to control motors, relays, sensors, and other actuators, making it a solid foundation for more complex IoT applications such as smart home automation, industrial monitoring, and remote device management.